LLM forecaster prompt experiments: synthesis

Madhav Khanal, with Jakub Kryś, Jeff Mohl, Matt Smith · SaferAI, Spring–August 2026 · 2026-08-30

1 Setup

- **Task.** Predict $p = \Pr[\text{AI model } m \text{ solves benchmark task } t \text{ in one attempt}]$, one prediction per (task, model) pair; ground truth is recorded pass/fail from a suite of cybersecurity benchmarks (Lyptus). Forecaster: Claude Sonnet 4.6, temperature 0, one call per pair, reporting percentiles $p_{25}/p_{50}/p_{75}$.
- **Evidence in the prompt.** Example tasks with their pass/fail outcomes and group pass rates, all drawn from difficulty levels *other than* the target's. The target's difficulty label is hidden from the model.
- **Splits** (fixed once). Search set: 84 pairs, used to select prompts. Held-out set: 230 pairs (21 unseen tasks × 11 models), used only for final scoring. Two further benchmarks reserved for a single future test.
- **Metrics.** Brier $(p_{50} - y)^2$ and CRPS of a Beta fitted to the three percentiles; lower is better. Noise floor from re-running the same prompt: sd 0.0065 per single search-set pass, ≈ 0.002 per multi-pass held-out estimate.
- **Reference points, held-out set.** Minimal hand-written prompt (the *seed*): 0.1312. *Difficulty-oracle table*: predict m's mean pass rate over the other tasks at t's difficulty level (leave-one-out); this no-LLM lookup reads the label the model never sees: 0.1034.

2 The 82 prompts and where they came from

Prompts come from GEPA, an evolutionary search that mutates a prompt with an LLM rewriter and keeps mutations that score better (arXiv:2507.19457). Every claim below uses multi-pass held-out scores: in both search runs, the prompt that looked best on the search set did not hold up on the held-out set.

source	prompts	best held-out gain	note
minimal hand-written prompt (*seed*)	1	reference	task statement and output format, no guidance
evolutionary search, run 1 (ran with a scoring bug: unparsable answers were scored as maximally wrong)	25	+0.027⋆ (8/8 paired passes)	three of its prompts verified as real gains after the fix
evolutionary search, run 2 (bug fixed, otherwise identical)	25	+0.016 (3/3)	independent replication of the method
two search variants (wider acceptance test; rewriter told to avoid numeric content)	32	none beat the seed	the no-numbers variant serves as a natural control
single-feature prompts, hand-built for §3	2	+0.020 / +0.006	seed plus exactly one feature each

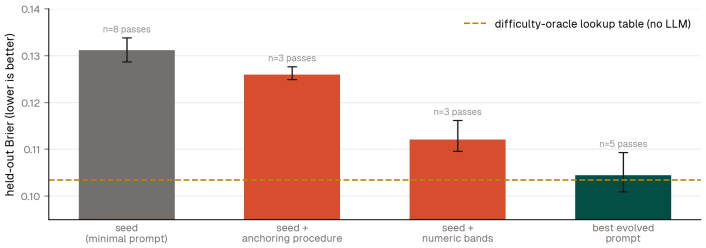
⋆best evolved prompt; used in the figure below.

3 Which prompt features matter

Every evolved prompt was tagged for five recurring features (deterministic keyword rubric). Features arrive in bundles, so the tag screen is directional; the intervention below is the causal test.

feature	what the prompt text says	n/82	ΔBrier	held-out evidence
anchoring instruction	base the probability on the observed pass rates in the evidence	82	—	in every evolved prompt
nearest-example weighting	if one example closely matches the target, weight its outcome above its group average	31	−0.0043	in all winners; absent from the no-numbers control
numeric probability bands	explicit ranges per task kind, e.g. trivial lookup 0.92–0.99, live exploitation 0.10–0.30	45	−0.0008	in all winners; absent from control; causal test below
anti-hedging	do not default to 0.5 when unsure	46	−0.0025	also in the control's null prompts: not sufficient
task-type adjustment	shift down for tasks against a live service, up for static file analysis	69	−0.0029	near-universal: not sufficient

Causal test. Each half of the best evolved prompt was added to the seed alone and scored on the held-out set, 3 paired passes each. The numeric bands alone recover most of the full prompt's gain; the same recipe with every number removed recovers little. CRPS agrees (seed 0.216, bands 0.189, best evolved 0.178).



4 Conclusions

1. Put numbers in the prompt; asking the model to derive them does not substitute. An explicit numeric anchor in the prompt carries most of the best prompt's gain; the same recipe with the numbers removed carries a third of it. This matches the project's oldest finding, that elicited values track whatever number the prompt provides, and turns that bias into the working mechanism. Prompt design for forecasting is deciding *which numbers the model sees*, not writing better reasoning steps. **2. Prompt optimization is real, bounded, and stops at a lookup table.** Two independent runs improved the forecaster, entirely by improving discrimination (it was already calibrated). The optimized model matches, and never beats, the difficulty-oracle table; the strongest base model reaches the same point with a hand-written prompt. In distribution the table is the ceiling, and optimization is worth about one model generation. **3. What is left is transfer.** A table cannot extrapolate to new benchmarks; a model might. The reserved out-of-distribution test decides whether the learned recipe has value beyond the table.

Reproducibility. Scripts `feature_report_data.py`, `val_noise_study.py`, `parse_failure_audit.py` (gepa repo); raw runs on `results/*` branches; curated data, prompt texts, this document: `experiments/III_gepa_optimization/summary/`; earlier stages: `prompt_sensitivity/`, experiments I/II, final report §3.